

Todos Application

1. HTML Input Element

1.1 Placeholder

Placeholder is the text that appears in the HTML `input` element when no value is set. We can specify it using the HTML attribute `placeholder`.

```
<input type="text" placeholder="Enter your name" />
```

2. JavaScript Built-in Functions

2.1 `alert()`

The `alert()` function displays an alert box with a specified message and an OK button.

```
alert("Enter Valid Text");
```

3. DOM Properties

3.1 Checked

The `checked` property sets or returns the checked status of an HTML checkbox `input` element as a boolean value

```
let checkboxElement = document.getElementById(checkboxId);
```

```
checkboxElement.checked = true;
```

4. DOM Manipulations

4.1 The removeChild() Method

The `removeChild()` method removes an HTML child element of the specified HTML parent element from the DOM and returns the removed HTML child element.

```
function onDeleteTodo(todoId) {  
  
    let todoElement = document.getElementById(todoId);  
  
    todoItemsContainer.removeChild(todoElement);  
  
}
```

4.2 The classList.toggle() Method

The `classList.toggle()` method is used to toggle between adding and removing a class name from an HTML element.

```
function onTodoStatusChange(checkboxId, labelId) {  
  
    let checkboxElement = document.getElementById(checkboxId);  
  
    let labelElement = document.getElementById(labelId);  
  
    labelElement.classList.toggle('checked');  
  
}
```

We can replace `classList.add()` and `classList.remove()` methods with `classList.toggle()` method.

5. Execution Context

The environment in which JavaScript Code runs is called Execution Context.

Execution context contains all the variables, objects, and functions.

Execution Context is destroyed and recreated whenever we reload an Application.

6. Storage Mechanisms

6.1 Client-Side Data Storage

Client-Side Data Storage is storing the data on the client (user's machine).

- Local Storage
- Session Storage
- Cookies
- IndexedDB and many more.

6.2 Server-Side Data Storage

Server-Side Data Storage is storing the data on the server.

7. Local Storage

It allows web applications to store data locally within the user's browser.

It is a Storage Object. Data can be stored in the form of key-value pairs.

Please note that both key and value must be strings. If their type is other than a string, they get converted to strings automatically.

fullName	Rahul Attuluri
gender	Male
city	Delhi

To access and work with Local Storage we have below methods:

- **setItem()**
- **getItem()**
- **clear()**
- **removeItem()**

7.1 The setItem() Method

The setItem() method can be used for storing data in the Local Storage.

Syntax:

```
localStorage.setItem("Key", "Value");
```

7.2 The getItem() Method

The getItem() method can be used for getting data from the Local Storage.

Syntax: `localStorage.getItem("Key");`

8. Values

8.1 null

We use null in a situation where we intentionally want a variable but don't need a value to it.

```
let selectedColor = null;
```

```
console.log(selectedColor); // null
```

```
console.log(typeof(selectedColor)); // object
```

9. HTML Elements

9.1 The textarea Element

The HTML textarea element can be used to write the multiline text as an input.

```
<textarea rows="8" cols="55"></textarea>
```

- The HTML rows attribute specifies the number of lines.
- The HTML cols attribute specifies the number of characters per each line.

10. JavaScript Object Notation (JSON)

JSON is a data representation format used for:

- Storing data (Client/Server)

- Exchanging data between Client and Server

10.1 Supported Types

- Number
- String
- Boolean
- Array
- Object
- Null

10.2 JS Object vs JSON Object

In JSON, all keys in an object must be enclosed with double-quotes. While in JS, this is not necessary.

JS: `let profile = {`

`name: "Rahul",`

`age: 29,`

`designation: "Web Developer"`

`};`

JSON: `let profile = {`

`"name": "Rahul",`

`"age": 29,`

"designation": "Web Developer"

};

10.3 JSON Methods

10.3.1 JSON.stringify()

It converts the given value into JSON string.

Syntax: JSON.stringify(value)

10.3.2 JSON.parse()

It parses a JSON string and returns a JS object.

Syntax: JSON.parse(string)

11. Array Methods

Method	Functionality
includes, indexOf, lastIndexOf, find, findIndex()	Finding Elements
push, unshift, splice	Adding Elements
pop, shift, splice	Removing Elements
concat, slice	Combining & Slicing Arrays

join

Joining Array Elements

sort

Sorting Array Elements

11.1 splice()

The `splice()` method changes the contents of an array.

Using `splice()` method, we can

- Remove existing items
- Replace existing items
- Add new items

11.1.1 Removing existing items

Syntax: `arr.splice(Start, Delete Count)`

- **Start:** Starting Index
- **Delete Count:** Number of items to be removed, starting from the given index

```
let myArray = [5, "six", 2, 8.2];
```

```
myArray.splice(2, 2);
```

```
console.log(myArray); // [5, "six"]
```



```
let deletedItems = myArray.splice(2, 2);
```

```
console.log(deletedItems); // [2, 8.2]
```

The `splice()` method returns an array containing the deleted items.

11.1.2 Adding new items

Syntax: `arr.splice(Start, Delete Count, Item1, Item2 ... )`

Here the `Item1, Item2 ...` are the items to be added, starting from the given index.

```
let myArray = [5, "six", 2, 8.2];
```

```
myArray.splice(2, 0, "one", false);
```

```
console.log(myArray); // [5, "six", "one", false, 2, 8.2]
```

11.1.3 Replacing existing items

Syntax: `arr.splice(Start, Delete Count, Item1, Item2 ... )`

```
let myArray = [5, "six", 2, 8.2];
```

```
myArray.splice(2, 1, true);
```

```
console.log(myArray); // [5, "six", true, 8.2]
```

11.2 findIndex()

The `findIndex()` method returns the first item's index that satisfies the provided testing function. If no item is found, it returns `-1`.

Syntax: `arr.findIndex(Testing Function)`

Here Testing Function is a function to execute on each value in the array.

```
let myArray = [5, 12, 8, 130, 44];

let itemIndex = myArray.findIndex(function(eachItem) {

    console.log(eachItem);

});
```

In the above code snippet, the below function is a Testing Function.

```
function(eachItem) { console.log(eachItem); }
```

11.3 includes()

The `includes()` method returns `true` if the provided item exists in the array. If no item is found, it returns `false`.

Syntax: `arr.includes(item)`

11.4 indexOf()

The `indexOf()` method returns the first index at which a given item can be found in the array. If no item is found, it returns `-1`.

Syntax: `arr.indexOf(item)`

11.5 lastIndexOf()

The `lastIndexOf()` method returns the last index at which a given item can be found in the array. If no item is found, it returns -1.

Syntax: `arr.lastIndexOf(item)`

11.6 find()

The `find()` method returns the first item's value that satisfies the provided testing function. If no item is found, it returns undefined.

Syntax: `arr.find(Testing Function)`

1.7 unshift()

The `unshift()` method adds one or more items to the beginning of an array and returns the new array length.

Syntax: `arr.unshift(item1,item2, ..., itemN)`

11.8 shift()

The `shift()` method removes the first item from an array and returns that removed item.

Syntax: `arr.shift()`

11.9 concat()

The `concat()` method can be used to merge two or more arrays.

This method does not change the existing arrays but instead returns a new array.

Syntax: `let newArray = arr1.concat(arr2);`

11.10 slice()

The `slice()` method returns a portion between the specified start index and end index(end index not included) of an array into a new array.

Syntax: `arr.slice(startIndex, endIndex)`

11.11 join()

The `join()` method creates and returns a new string by concatenating all of the items in an array, separated by commas or a specified separator string.

If the array has only one item, then it will be returned without using the specified separator.

Syntax: `arr.join(separator)`

Here separator is a string used to separate each item of the array. If omitted, the array items are separated with a comma.

11.12 sort()

The `sort()` method sorts the items of an array and returns the sorted array.
The default sort order is ascending.

Syntax:`arr.sort()`

12. Local Storage

12.1 The `removeItem()` Method

The `removeItem()` method removes the specified storage object item based on the key.

Syntax:`localStorage.removeItem(key)`

Key - Name of the key to be removed

```
localStorage.removeItem("todoList");
```